Arquitetura de Banco de Dados para Ordem Paranormal RPG App

Introdução

Este documento detalha a arquitetura de banco de dados proposta para um aplicativo de criação e gerenciamento de personagens para o RPG **Ordem Paranormal**, incorporando as regras do livro base e do suplemento **Sobrevivendo ao Horror**. O objetivo é estabelecer um modelo de dados robusto e flexível, capaz de suportar cálculos em tempo real e facilitar a criação de fichas de personagem interativas.

1. Entidades de Referência (Dados Estáticos)

As entidades de referência representam os dados fundamentais do sistema de jogo que são predefinidos e não se alteram durante a utilização do aplicativo pelo usuário. Estes dados são essenciais para a validação e a apresentação das opções disponíveis na criação de personagens.

1.1. Tabela: `classes`

Esta tabela armazena as informações básicas das três classes principais do jogo: Combatente, Especialista e Ocultista. Cada classe possui características iniciais distintas que influenciam diretamente os atributos e recursos do personagem.

Campo	Tipo de Dado	Descrição	Observações
`id`	UUID	Identificador único da classe.	Chave primária.
`nome`	String	Nome da classe (ex: Combatente, Especialista, Ocultista).	
`pv_inicial`	Integer	Pontos de Vida iniciais concedidos pela classe.	
`pv_por_nivel`	Integer	Pontos de Vida ganhos a cada Nível/NEX.	

`pe_inicial`	Integer	Pontos de Esforço iniciais concedidos pela classe.	
`pe_por_nivel`	Integer	Pontos de Esforço ganhos a cada Nível/NEX.	
`san_inicial`	Integer	Pontos de Sanidade iniciais concedidos pela classe.	
`san_por_nivel`	Integer	Pontos de Sanidade ganhos a cada Nível/NEX.	
`pericias_treinadas_base`	Integer	Número de perícias treinadas que a classe concede inicialmente.	

1.2. Tabela: `pericias`

Contém a lista de todas as perícias disponíveis no sistema, juntamente com o atributo base associado a cada uma e a indicação se a perícia pode ser utilizada sem treinamento.

Campo	Tipo de Dado	Descrição	Observações
`id`	UUID	Identificador único da perícia.	Chave primária.
`nome`	String	Nome da perícia (ex: Acrobacia, Atletismo, Ocultismo).	
`atributo_base`	Enum (String)	Atributo principal que governa a perícia (FOR, AGI, INT, VIG, PRE).	
`somente_treinada`	Boolean	Indica se a perícia exige treinamento para ser utilizada.	`TRUE` para perícias como Ocultismo.

1.3. Tabela: `origens`

Armazena as diversas origens que um personagem pode ter, cada uma conferindo bônus específicos, perícias treinadas e, em alguns casos, um poder único. O suplemento **Sobrevivendo ao Horror** expande significativamente esta lista.

Campo	Tipo de Dado	Descrição	Observações
`id`	UUID	Identificador único da origem.	Chave primária.
`nome`	String	Nome da origem (ex: Acadêmico, Agente de Saúde, Colegial).	
`descricao`	Text	Descrição detalhada da origem e seus benefícios.	
`pericias_concedidas`	JSON Array	Lista de IDs de perícias que a origem concede como treinadas.	Referência a `pericias.id`.
`poder_concedido`	String	Nome ou ID do poder especial concedido pela origem.	Pode ser `NULL` se não houver poder.

1.4. Tabela: `rituais`

Esta tabela lista todos os rituais disponíveis, incluindo seu elemento, círculo, custo de PE e outras propriedades que afetam sua conjuração e efeitos. A complexidade dos rituais é um ponto central para os Ocultistas.

Campo	Tipo de Dado	Descrição	Observações
`id`	UUID	Identificador único do ritual.	Chave primária.
`nome`	String	Nome do ritual (ex: Descarnar, Terceiro Olho).	
`elemento`	Enum (String)	Elemento paranormal	

		associado (Sangue, Morte, Conhecimento, Energia, Medo).	
`circulo`	Integer	Círculo do ritual (1, 2, 3 ou 4).	Define o nível de poder.
`execucao`	String	Tempo de execução do ritual (Padrão, Movimento, Completa).	
`alcance`	String	Alcance do ritual (Curto, Médio, Longo, Pessoal, Toque).	
`alvo`	String	Descrição do alvo do ritual (ex: 1 criatura, área, você).	
`duracao`	String	Duração do efeito do ritual (Instantânea, Cena, Rodadas, Permanente).	
`resistencia`	String	Tipo de teste de resistência (Vontade, Fortitude, Reflexos).	Pode ser `NULL` se não houver resistência.
`custo_pe`	Integer	Custo base em Pontos de Esforço para conjurar o ritual.	
`descricao`	Text	Descrição completa do ritual e seus efeitos.	

1.5. Tabela: `itens`

Contém informações sobre todos os itens que podem ser encontrados ou comprados, incluindo armas, equipamentos, e itens amaldiçoados. Esta tabela é crucial para o gerenciamento de inventário e carga.

Campo	Tipo de Dado	Descrição	Observações

`id`	UUID	Identificador único do item.	Chave primária.
`nome`	String	Nome do item (ex: Pistola, Kit Médico, Amuleto da Sorte).	
`tipo`	Enum (String)	Tipo do item (Arma, Proteção, Utilitário, Amaldiçoado, etc.).	
`descricao`	Text	Descrição do item e seus efeitos.	
`espacos`	Float	Quantidade de espaços que o item ocupa no inventário.	Pode ser fracionado (ex: 0.5).
`categoria`	Enum (String)	Categoria do item (0, I, II, III, IV).	Define a raridade/poder.
`peso`	Float	Peso do item em kg.	Alternativa ou complemento a `espacos`.
`preco`	Integer	Preço base do item em moeda do jogo.	
`requisitos`	JSON	Requisitos para usar o item (ex: Força 2, NEX 10%).	Pode ser `NULL`.

2. Entidades do Personagem (Dados do Usuário)

Estas tabelas armazenam os dados dinâmicos e específicos de cada personagem criado pelos usuários do aplicativo. Elas são projetadas para refletir o estado atual e a progressão individual de cada ficha.

2.1. Tabela: `personagens`

Representa a ficha principal do personagem, ligando-o ao usuário e às suas escolhas de classe e origem. Contém os dados mais voláteis, como PV, PE e Sanidade atuais.

Campo	Tipo de Dado	Descrição	Observações

`id`	UUID	Identificador único do personagem.	Chave primária.
`user_id`	UUID	Chave estrangeira para a tabela de usuários.	Permite que o usuário tenha múltiplos personagens.
`nome`	String	Nome do personagem.	
`classe_id`	UUID	Chave estrangeira para a tabela `classes`.	
`origem_id`	UUID	Chave estrangeira para a tabela `origens`.	
`nivel`	Integer	Nível mundano do personagem (1-20).	Introduzido por **Sobrevivendo ao Horror**.
`nex`	Integer	Nível de Exposição Paranormal (0-99%).	
`patente`	Enum (String)	Patente do personagem na Ordo Realitas (Recruta, Operador, Agente Especial, etc.).	
`pv_atual`	Integer	Pontos de Vida atuais do personagem.	
`pe_atual`	Integer	Pontos de Esforço atuais do personagem.	
`san_atual`	Integer	Pontos de Sanidade atuais do personagem.	
`san_max_bonus`	Integer	Bônus/penalidade manual na Sanidade máxima.	Para ajustes específicos do mestre.

2.2. Tabela: `personagem_atributos`

Armazena os valores dos cinco atributos do personagem. Esta tabela é separada da principal para manter a normalização e facilitar futuras expansões.

Campo	Tipo de Dado	Descrição	Observações
`personagem_id`	UUID	Chave estrangeira para a tabela `personagens`.	Chave primária composta com `atributo_id`.
`forca`	Integer	Valor do atributo Força.	
`agilidade`	Integer	Valor do atributo Agilidade.	
`intelecto`	Integer	Valor do atributo Intelecto.	
`vigor`	Integer	Valor do atributo Vigor.	
`presenca`	Integer	Valor do atributo Presença.	

2.3. Tabela: `personagem_pericias`

Registra o grau de treinamento do personagem em cada perícia. Permite rastrear se uma perícia é Destreinada, Treinada, Veterana ou Expert.

Campo	Tipo de Dado	Descrição	Observações
`personagem_id`	UUID	Chave estrangeira para a tabela `personagens`.	Chave primária composta com `pericia_id`.
`pericia_id`	UUID	Chave estrangeira para a tabela `pericias`.	

`grau`	Enum (String)	Grau de treinamento (Destreinado, Treinado, Veterano, Expert).	Mapeia para bônus (0, 5, 10, 15).

2.4. Tabela: `personagem_inventario`

Gerencia os itens que o personagem possui, incluindo a quantidade e os espaços que ocupam. Esta tabela é crucial para a mecânica de carga e recursos contados de **Sobrevivendo ao Horror**.

Campo	Tipo de Dado	Descrição	Observações
`id`	UUID	Identificador único do item no inventário.	Chave primária.
`personagem_id`	UUID	Chave estrangeira para a tabela `personagens`.	
`item_id`	UUID	Chave estrangeira para a tabela `itens`.	
`quantidade`	Integer	Quantidade do item no inventário.	
`espacos_ocupados`	Float	Total de espaços ocupados por este item (quantidade * item.espacos).	Campo calculado.

2.5. Tabela: `personagem_rituais`

Armazena os rituais que o personagem conhece e pode conjurar. Pode incluir informações adicionais sobre variações ou modificações específicas que o personagem aprendeu.

Campo	Tipo de Dado	Descrição	Observações
`personagem_id`	UUID	Chave estrangeira para a tabela `personagens`.	Chave primária composta com `ritual_id`.

`ritual_id`	UUID	Chave estrangeira para a tabela `rituais`.	
`variacao`	Enum (String)	Variação do ritual (Indouta, Discente, Verdadeira).	
`modificacoes`	JSON Array	Lista de modificações aplicadas ao ritual.	Pode ser `NULL`.

3. Lógica de Cálculos em Tempo Real (Regras de Negócio)

Para garantir que o aplicativo seja uma ferramenta útil na mesa de jogo, os seguintes cálculos devem ser implementados para serem atualizados dinamicamente com base nas escolhas do jogador e na progressão do personagem. A separação de Nível e NEX, introduzida por **Sobrevivendo ao Horror**, é fundamental para estes cálculos.

3.1. Pontos de Vida (PV Máximo)

O PV máximo de um personagem é uma combinação da base da classe, do atributo Vigor e da progressão por Nível e NEX. A fórmula é:

Plain Text

```
PV_Max = Classe.pv_inicial + Personagem.vigor + ((Personagem.nivel + Personagem
```

- **`Classe.pv_inicial`**: Valor inicial de PV definido pela classe (ex: Combatente = 20).
- **`Personagem.vigor`**: Valor atual do atributo Vigor do personagem.
- **`Personagem.nivel`**: Nível mundano do personagem (1-20).
- **`Personagem.nex_bonus`**: Bônus de NEX para cálculos de progressão (pode ser `Personagem.nex / 5` ou similar, dependendo da regra exata de como NEX se traduz em "nível" para PV/PE/SAN no suplemento).
- **`Classe.pv_por_nivel`**: PV ganhos por Nível/NEX, definido pela classe (ex: Combatente = 4).

3.2. Pontos de Esforço (PE Máximo)

Similar aos PV, o PE máximo é influenciado pela classe, pelo atributo Presença e pela progressão.

Plain Text

```
PE_Max = Classe.pe_inicial + Personagem.presenca + ((Personagem.nivel + Persona
```

- **Classe.pe_inicial** : Valor inicial de PE definido pela classe (ex: Ocultista = 6).
- **Personagem.presenca** : Valor atual do atributo Presença do personagem.
- **Personagem.nivel** : Nível mundano do personagem (1-20).
- **Personagem.nex_bonus** : Bônus de NEX para cálculos de progressão.
- **Classe.pe_por_nivel** : PE ganhos por Nível/NEX, definido pela classe (ex: Ocultista = 3).

3.3. Sanidade (SAN Máxima)

A Sanidade máxima reflete a resiliência mental do personagem, afetada pela classe, pela progressão e por quaisquer bônus ou penalidades adicionais.

Plain Text

```
SAN_Max = Classe.san_inicial + ((Personagem.nivel + Personagem.nex_bonus) * Cla
```

- **Classe.san_inicial** : Valor inicial de Sanidade definido pela classe (ex: Especialista = 16).
- **Personagem.nivel** : Nível mundano do personagem (1-20).
- **Personagem.nex_bonus** : Bônus de NEX para cálculos de progressão.
- **Classe.san_por_nivel** : Sanidade ganha por Nível/NEX, definido pela classe (ex: Especialista = 4).
- **Personagem.san_max_bonus** : Ajustes manuais na Sanidade máxima (positivo ou negativo).
- **Perda_por_Poderes** : Sanidade máxima perdida devido a poderes paranormais ou rituais permanentes.

3.4. Defesa

A defesa do personagem é um cálculo simples que considera a Agilidade, proteções e outros bônus.

Plain Text

```
Defesa = 10 + Personagem.agilidade + Protecoes_Equipadas + Outros_Bonus
```

- **Personagem.agilidade** : Valor atual do atributo Agilidade do personagem.
- **Protecoes_Equipadas** : Bônus de defesa fornecido por armaduras e outros itens de proteção.

- **`Outros_Bonus`** : Quaisquer outros bônus de defesa (ex: poderes, rituais).

3.5. Carga Máxima

A carga máxima determina quantos itens o personagem pode carregar antes de sofrer penalidades. A regra padrão é baseada na Força, mas pode haver variações.

```
Plain Text

Carga_Maxima = 5 + (Personagem.forca * 5)
```

- **`Personagem.forca`** : Valor atual do atributo Força do personagem.

3.6. Testes de Perícia

Os testes de perícia envolvem a rolagem de dados e a aplicação de bônus de treinamento. A lógica deve selecionar o maior resultado de `Xd20` e adicionar os bônus.

```
Plain Text

Resultado_Teste = Max(Rolar(Personagem.atributo_base_da_pericia)d20) + Personag
```

- **`Personagem.atributo_base_da_pericia`** : O valor do atributo associado à perícia que está sendo testada.
- **`Personagem_Pericias.grau_bonus`** : O bônus numérico correspondente ao grau de treinamento da perícia (0 para Destreinado, 5 para Treinado, 10 para Veterano, 15 para Expert).
- **`Outros_Bonus`** : Quaisquer outros bônus (ex: itens, poderes, rituais).

4. Considerações Adicionais para o Aplicativo

- **Relacionamentos:** As chaves estrangeiras (FK) são cruciais para manter a integridade dos dados e permitir consultas eficientes. Por exemplo, `personagens.classe_id` se relaciona com `classes.id`.
- **UUIDs:** A utilização de UUIDs (Universally Unique Identifiers) para chaves primárias garante unicidade global e facilita a fusão de dados de diferentes fontes, se necessário, além de ser mais seguro em ambientes distribuídos.
- **JSON Fields:** Campos JSON (como `pericias_concedidas` em `origens` ou `modificacoes` em `personagem_rituais`) oferecem flexibilidade para armazenar dados semi-estruturados que podem variar em complexidade sem a necessidade de criar novas tabelas para cada pequena variação.

- **Escalabilidade:** A separação entre dados estáticos (referência) e dinâmicos (personagem) otimiza o desempenho, pois os dados de referência podem ser cacheados e acessados rapidamente.
- **Regras de Negócio:** A lógica de cálculo deve ser implementada na camada de aplicação do software, garantindo que o banco de dados permaneça focado no armazenamento e recuperação de dados, enquanto a aplicação lida com a complexidade das regras do jogo.

Conclusão

Esta arquitetura de banco de dados fornece uma base sólida para o desenvolvimento de um aplicativo de gerenciamento de personagens para Ordem Paranormal RPG. Ao modelar cuidadosamente as entidades e prever a lógica de cálculo, o sistema será capaz de oferecer uma experiência fluida e precisa para os jogadores, facilitando a criação e o acompanhamento de suas fichas de personagem em tempo real. A inclusão das mecânicas de **Sobrevivendo ao Horror** garante que o aplicativo esteja atualizado com as últimas adições ao universo do jogo, proporcionando uma experiência completa e imersiva.